



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών (ΣΗΜΜΥ)

8ο εξάμηνο

Ψηφιακά Συστήματα VLSI

6η εργαστηριακή άσκηση (Bonus)

Δημήτρης Βατάλας

(Α.Μ.: 03121093)

Αριστοτέλης Γεώργιος Δρίτσας

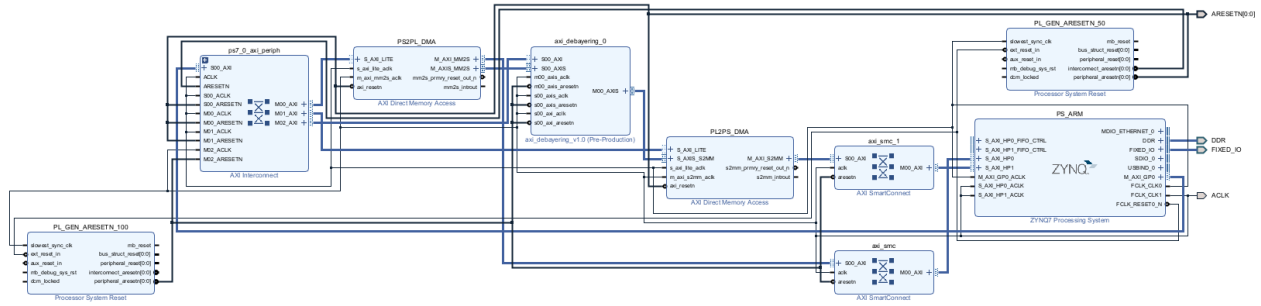
(Α.Μ.: 03122127)

Αθήνα, Μάιος 2026

Contents

1	Block Διάγραμμα Συστήματος	2
2	Σύντομη επεξήγηση των βασικών δομικών μονάδων του Κυκλώματος	2
2.1	Γενική Επισκόπηση	2
2.2	Serial to Parallel (Runtime Reconfigurable)	2
2.3	Calculator	2
2.4	FSM (Runtime Reconfigurable)	3
2.5	AXI-Lite Slave Interface	3
2.6	AXI-Stream Slave & Master Interfaces	3
2.7	reconfigurable_debayering_filter (Top Module)	3
3	Σύγκριση Πόρων FPGA: Runtime Reconfigurable vs Στατική Έκδοση	3
3.1	Στατική Έκδοση (5η Άσκηση)	3
3.2	Runtime Reconfigurable Έκδοση (6η Άσκηση)	4
3.3	Παρατηρήσεις	5
4	Σύγκριση Speedup CPU vs. FPGA	5
5	Waveform Προσομοίωσης	5

1 Block Διάγραμμα Συστήματος



2 Σύντομη επεξήγηση των βασικών δομικών μονάδων του Κυκλώματος

2.1 Γενική Επισκόπηση

Η 6η εργαστηριακή άσκηση αποτελεί επέκταση της 5ης, με κύριο στόχο τη μετατροπή του debayering φίλτρου σε *runtime reconfigurable*, δηλαδή σε ένα σύστημα που μπορεί να αλλάζει δυναμικά το μέγεθος εικόνας που υποστηρίζει χωρίς επαναπρογραμματισμό του FPGA.

Η βασική αλλαγή σε σχέση με την 5η άσκηση είναι ότι η generic παράμετρος N (που καθόριζε στατικά τη διάσταση της εικόνας κατά τη σύνθεση) αντικαταστάθηκε από ένα σήμα εισόδου `image_dim`, το οποίο καθορίζεται δυναμικά από τον ARM επεξεργαστή μέσω διεπαφής AXI-Lite. Παράλληλα, προστέθηκε το σήμα `image_dim_vld` που υποδεικνύει την εγκυρότητα της νέας τιμής.

2.2 Serial to Parallel (Runtime Reconfigurable)

Στο module `serial_to_parallel` έχουν πραγματοποιηθεί οι κατάλληλες αλλαγές, ώστε να υποστηρίζεται παραμετρικό μέγεθος εικόνας κατά τον χρόνο εκτέλεσης. Συγκεκριμένα, οι πράξεις που στο προηγούμενο εργαστήριο βασιζόνταν στη generic παράμετρο N αντικαταστάθηκαν από αντίστοιχες πράξεις με το σήμα `image_dim`, το οποίο λαμβάνεται μέσω της διεπαφής AXI-Lite.

Αξίζει να σημειωθεί ότι, ήδη από τις προηγούμενες εργασίες, το σύστημα υποστήριζε την επεξεργασία εικόνων διαφορετικών διαστάσεων, αρκεί η διάστασή τους να χωρούσε στη FIFO. Αυτό επιτυγχανόταν κάθε φορά μέσω κατάλληλου παραμετροποιημένου mapping που έχουμε αναλύσει σε προηγούμενες αναφορές. Επομένως, δεν απαιτήθηκε κάποια σημαντική αλλαγή στο συγκεκριμένο module, όπως θα συνέβαινε αν η λειτουργία αυτή δεν είχε ήδη υλοποιηθεί.

Το υπόλοιπο της λειτουργίας παραμένει ίδιο με την 5η Άσκηση. Σε κάθε κύκλο ρολογιού εισάγεται ένα νέο pixel στην πρώτη FIFO, προωθείται διαδοχικά στις επόμενες και σχηματίζεται ένα παράθυρο 3×3 , το οποίο χρησιμοποιείται από τον calculator.

2.3 Calculator

Ο calculator παραμένει ουσιαστικά αμετάβλητος σε σχέση με την 5η εργαστηριακή άσκηση. Δέχεται ως είσοδο το παράθυρο 3×3 γειτονικών pixels (`din00` έως `din22`), αναγνωρίζει τον τύπο κάθε pixel (GREEN1, GREEN2, BLUE, RED) βάσει του προτύπου GBRG και υπολογίζει τις τιμές R, G, B εφαρμόζοντας τους κατάλληλους μέσους όρους.

2.4 FSM (Runtime Reconfigurable)

Στο συγκεκριμένο module όπως και σε όλα τα υπόλοιπα πρακτικά, δεν απαιτήθηκαν ουσιαστικές αλλαγές στη βασική λειτουργία του. Η μόνη διαφοροποίηση αφορά την αντικατάσταση της *generic* μεταβλητής, που χρησιμοποιούνταν στο προηγούμενο εργαστήριο για τον καθορισμό του μεγέθους της εικόνας, με το σήμα `image_dim`. Με αυτόν τον τρόπο, η διάσταση της εικόνας μπορεί πλέον να καθορίζεται δυναμικά κατά τον χρόνο εκτέλεσης, χωρίς να απαιτείται επανασύνθεση του υλικού.

2.5 AXI-Lite Slave Interface

Η διεπαφή AXI-Lite Slave είναι υπεύθυνη για τη μεταφορά του μεγέθους εικόνας από τον ARM επεξεργαστή στο FPGA. Ο ARM γράφει την επιθυμητή τιμή του `image_dim` σε έναν καταχωρητή της AXI-Lite διεπαφής, η οποία στη συνέχεια παράγει τα σήματα `image_dim` και `image_dim_vld` για το debayering φίλτρο. .

2.6 AXI-Stream Slave & Master Interfaces

Οι διεπαφές AXI-Stream παραμένουν ίδιες με αυτές της 5ης εργαστηριακής άσκησης:

- **AXI-Stream Slave:** Λαμβάνει τα pixels εισόδου από τον DMA. Το `Slave_TDATA[7:0]` συνδέεται στο `pixel[7:0]`, το `Slave_TVALID` στο `valid_in`.
- **AXI-Stream Master:** Αποστέλλει τα αποτελέσματα RGB στον DMA. Το `Master_TDATA[31:0] = "00000000" & R[7:0] & G[7:0] & B[7:0]`, το `Master_TVALID = valid_out` και το `Master_TLAST = image_finished`.

2.7 reconfigurable_debayering_filter (Top Module)

Το top module του συστήματος ενσωματώνει όλα τα παραπάνω υποσυστήματα: τον `image_dim` decoder, την FSM, τον `serial_to_parallel` converter, τον calculator και τις AXI διεπαφές. Δεν περιέχει επιπλέον επεξεργαστική λογική, αλλά λειτουργεί αποκλειστικά ως επίπεδο διασύνδεσης.

Τα σήματα εισόδου περιλαμβάνουν: `clk`, `rst_n`, `new_image`, `valid_in`, `pixel[7:0]`, `image_dim` και `image_dim_vld`. Τα σήματα εξόδου περιλαμβάνουν: `R[7:0]`, `G[7:0]`, `B[7:0]`, `valid_out` και `image_finished`.

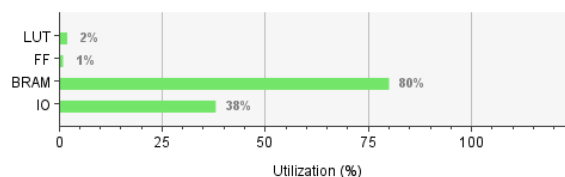
3 Σύγκριση Πόρων FPGA: Runtime Reconfigurable vs Στατική Έκδοση

Στον παρακάτω πίνακα παρουσιάζεται η σύγκριση της χρήσης πόρων FPGA μεταξύ της runtime reconfigurable έκδοσης (παρούσα άσκηση) και της στατικής έκδοσης (5η άσκηση, με generics) για διάφορα μεγέθη εικόνας, μετά το στάδιο Place & Route.

3.1 Στατική Έκδοση (5η Άσκηση)

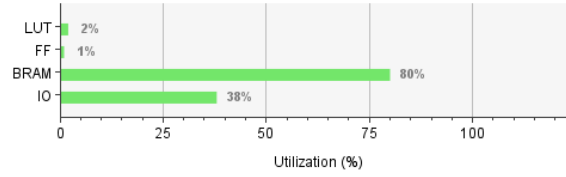
- N = 256

Resource	Utilization	Available	Utilization %
LUT	417	17600	2.37
FF	379	35200	1.08
BRAM	48	60	80.00
IO	38	100	38.00



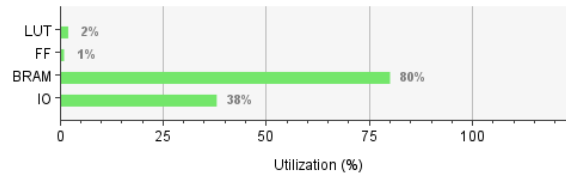
- $N = 512$

Resource	Utilization	Available	Utilization %
LUT	417	17600	2.37
FF	379	35200	1.08
BRAM	48	60	80.00
IO	38	100	38.00



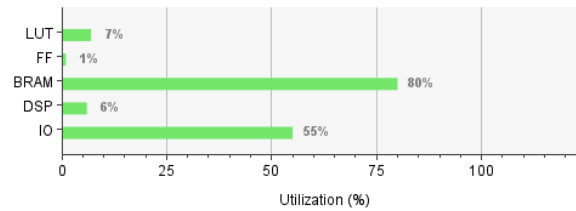
- $N = 1024$

Resource	Utilization	Available	Utilization %
LUT	417	17600	2.37
FF	379	35200	1.08
BRAM	48	60	80.00
IO	38	100	38.00



3.2 Runtime Reconfigurable Έκδοση (6η Άσκηση)

Resource	Utilization	Available	Utilization %
LUT	1190	17600	6.76
FF	395	35200	1.12
BRAM	48	60	80.00
DSP	5	80	6.25
IO	55	100	55.00



3.3 Παρατηρήσεις

Παρατηρούμε ότι η *runtime reconfigurable* έκδοση παρουσιάζει σταθερή χρήση πόρων, ανεξάρτητα από το μέγεθος της εικόνας που επεξεργάζεται. Αυτό συμβαίνει επειδή το υλικό παραμένει προφανώς αμετάβλητο. Η σχετική αύξηση που παρατηρείται σε σχέση με τη χρήση πόρων στην Άσκηση 5 οφείλεται στην προσθήκη της επικοινωνίας μέσω AXI_Lite, καθώς και στη μικρή τροποποίηση της αρχιτεκτονικής, όπως αναφέρθηκε προηγουμένως, ώστε να υποστηρίζεται η αλλαγή του μεγέθους της εικόνας κατά τον χρόνο εκτέλεσης.

Σημειώνεται επίσης ότι έχουμε το ίδιο utilization για όλα τα μεγέθη εικόνων και της 5ης άσκησης καθώς είχαμε χρησιμοποιήσει μια fifo σταθερού μεγέθους και σε εκείνη την υλοποίηση.

Το σημαντικό πλεονέκτημα της *runtime reconfigurable* έκδοσης είναι ότι με ένα μόνο bitstream μπορούμε να υποστηρίξουμε όλα τα μεγέθη εικόνων, γεγονός που καθιστά την προσέγγιση ιδανική για πραγματικές εφαρμογές όπου το μέγεθος εικόνας μεταβάλλεται δυναμικά.

4 Σύγκριση Speedup CPU vs. FPGA

Μετρήσαμε το χρόνο εκτέλεσης του debayering φίλτρου τόσο σε software (ARM επεξεργαστής) όσο και στο FPGA, για όλα τα υποστηριζόμενα μεγέθη εικόνων. Οι μετρήσεις πραγματοποιήθηκαν στο Zybo με clock 100 MHz για το FPGA.

εικόνας	CPU cycles	FPGA cycles	Speedup
256×256	2300313	341895	6.73
512×512	9205303	1367866	6.73
1024×1024	36802087	5453612	6.74

Αξίζει να σημειωθεί ότι για εικόνες μικρότερων διαστάσεων το παρατηρούμενο speedup ήταν αισθητά μικρότερο, ενώ για πολύ μικρές εικόνες έφτανε να είναι σχεδόν αμελητέο. Αυτό το αποδώσαμε στο συνδυασμό δύο παραγόντων, πιο συγκεκριμένα αρχικά οι εικόνες μικρών διαστάσεων είναι πιθανό να χωράνε εξ ολοκλήρου σε cache μνήμες κοντά στον πυρήνα του ARM, μειώνοντας έτσι δραστικά τον χρόνο εκτέλεσης του προγράμματος σε αυτόν, ενώ ταυτόχρονα, το επιπλέον overhead που εισάγει η επικοινωνία μέσω των AXI bus και των DMA transfers παραμένει σταθερό ανεξαρτήτως μεγέθους εικόνων. Έτσι ο συνδυασμός αυτών των δύο παραγόντων καθιστά ενδεχομένως τη χρήση του FPGA accelerator μη συμφέρουσα για μικρές εικόνες.

5 Waveform Προσομοίωσης

Παρακάτω παρουσιάζεται το waveform της προσομοίωσης του *runtime reconfigurable* debayering φίλτρου, που επαληθεύει την ορθή λειτουργία του κυκλώματος. Μπορεί να παρατηρήσει κανείς τη φάση ρύθμισης, κατά την οποία το `image_dim_vld` τίθεται σε '1' και καθορίζεται το μέγεθος εικόνας, ακολουθούμενη από τη φάση λήψης pixels (`valid_in`) και τελικά από τη φάση παραγωγής έγκυρης εξόδου (`valid_out`).

